claude-windows / fa4dc4eb-4705-47fd-b044-4f017b586b15

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\fa4dc4eb-4705-47fd-b044-4f017b586b15\subagents\workflows\wf_b05b902c-a7c\agent-ab60fa49c5987b62e.jsonl`
- SHA-256: `383eb8a35020b5bdb0533d375ffc0f07456a7f6ec854f950ef02fe2cb9d385d1`
- Source modified: `2026-06-19T16:32:44+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_b05b902c-a7c`
- session_id: `fa4dc4eb-4705-47fd-b044-4f017b586b15`

Transcript

1. user (2026-06-19T16:29:39.642Z)

Adversarially VERIFY this prominent-court-gate result for the multicourt clip GX010128. Independently re-run the measurement (recipe below) and check: (1) the polygon is RECALL-SAFE (pct_golden_in_polygon >= 0.85) and excludes a genuine adjacent court (not just drawn to match golden); (2) the precision/F1 gain is REAL and the dropped rallies are genuinely FALSE POSITIVES (matched no golden), not real rallies; (3) recall did NOT drop materially; (4) no number is inflated. If the reported result is wrong or inflated, return the CORRECTED row; if recall was silently lost, fix delta_recall + read. Output the verified/corrected row.

REPORTED ROW:

{
 "name": "GX010128",
 "n_gt": 52,
 "pct_golden_in_polygon": 1,
 "polygon_summary": "Normalized bbox quad [(0.15,0.0),(0.78,0.0),(0.78,0.70),(0.15,0.70)]. Derived from the 2D distribution of 27133 visible shuttle points: dominant foreground/prominent-court mass sits at x in [0.4,0.7], y in [0.2,0.6]; a tightly-stuck secondary cluster (~1685 pts at x~0.85, y~0.30-0.40 = adjacent court / stuck tracker) is excluded by capping x<=0.78. Box padded generously (x>=0.15, y up to 0.70) to stay recall-safe. Excludes 2070 visible pts (the secondary cluster) while retaining all 52 golden-rally medians.",
 "base_n_pred": 81,
 "base_precision": 0.493827,
 "base_recall": 0.769231,
 "base_f1": 0.601504,
 "base_tolf1": 0.466165,
 "gated_n_pred": 81,
 "gated_precision": 0.493827,
 "gated_recall": 0.769231,
 "gated_f1": 0.601504,
 "gated_tolf1": 0.466165,
 "n_dropped": 0,
 "n_dropped_were_fp": 0,
 "delta_precision": 0,
 "delta_recall": 0,
 "delta_f1": 0,
 "read": "NULL result: at min_in_court_frac=0.5 the gate is an exact no-op on GX010128 (0 dropped, P/R/F1 unchanged). Courts DO separate cleanly (a tight x~0.85 secondary cluster is excluded, polygon is fully recall-safe at 100% golden medians), but the min in-court fraction across all 81 windows is 0.62 -- no baseline window is majority-out-of-court, so the secondary-court points never accumulate into a droppable rally. No win, no harm; the over-fire (81 preds vs 52 gt, 41 FPs) is real but these FPs aren't out-of-court-majority."
}

RECIPE:
You are measuring the prominent-court shuttle gate's quality win on ONE multicourt badminton

```
golden clip, against human golden labels. Work in the repo root; run Python via Bash. Be
numerically exact ? do NOT estimate.

DATA
- Golden rallies: json.load(open("output/GX010128_golden_features.json"))["gts"] -> list of
[start,end] seconds.
- Manifest output/human_lovo_manifest.json: the entry with name==GX010128 gives trajectory
(path), fps, frame_width (=1920 for these; so frame_height=1080).

STEP 1 baseline (ungated) local rallies:
  from backend.eval.rally_seg_eval import windows_for
  from backend.eval.windowing import SERVED
  video={"name":NAME,"trajectory":TRAJ,"fps":FPS,"frame_width":FW,"gts":GTS}
  baseline = windows_for(video, SERVED)

STEP 2 derive a normalized (0-1) prominent-court polygon, RECALL-SAFE:
  from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
  points = TrackNetRunner.parse_trajectory_csv(TRAJ)   # each has .visible, .x, .y, .frame
  fh = 1080.0
  Inspect the 2D distribution of visible (x/FW, y/FH). Multicourt clips can have multiple
shuttle clusters (courts); the PROMINENT court = the dominant/foreground cluster (most shuttle
mass; foreground tends lower/larger in frame). Draw a polygon (quad or bounding box in
normalized coords) around the prominent cluster, EXCLUDING any clearly-separated secondary
(adjacent) cluster.
  VALIDATE recall-safety: for each golden rally compute its shuttle MEDIAN (x_norm,y_norm) over
frames in [s*FPS, e*FPS]; your polygon MUST contain >= 0.85 of golden-rally medians
(point_in_polygon from backend.pipeline.detectors.fusion_features). If < 0.85 widen it and re-
check; iterate. Record pct_golden_in_polygon. If the clip has NO clear secondary cluster to
exclude (courts overlap fully), say so ? a near-full-frame polygon is then a near no-op (an
honest null).

STEP 3 gate + score (numerically exact):
  from backend.pipeline.detectors.court_gate import gate_intervals_to_court
  gated = gate_intervals_to_court(baseline, points, polygon, FPS, FW, fh, min_in_court_frac=0.5)
  from backend.eval.metrics import score, match_intervals
  from backend.eval.tolerance_metrics import match_1to1
  b = score(baseline, GTS, 0.5); g = score(gated, GTS, 0.5)   # .precision/.recall/.f1
  base_tolf1 = match_1to1(baseline, GTS)[3]; gated_tolf1 = match_1to1(gated, GTS)[3]
  Of the DROPPED rallies (baseline minus gated), count how many were baseline FALSE POSITIVES: a
baseline pred is a FP if it is NOT in match_intervals(baseline, GTS, 0.5).matches as a matched
pred index.
  seg_ratio = n_pred / n_gt.

STEP 4 return the structured row. The win is REAL only if precision/F1 RISE with recall ~FLAT
(you cut FPs, not real rallies). If recall drops materially, or the clip barely over-fires, or
courts don't separate -> say so honestly (a null/negative result is a valid, valuable finding).
pct_golden_in_polygon < 0.85 means your polygon is unsafe ? fix it before reporting.
```

2. assistant (2026-06-19T16:29:44.739Z)

I'll verify this prominent-court-gate result adversarially. Let me start by understanding the data and code, then independently re-run the measurement.

3. user (2026-06-19T16:29:49.261Z)

```
error: cannot lock ref 'refs/remotes/origin/master': is at
69fb2f1417f12d882cf6ecc115361e75880f7311 but expected e358b285380956d18e7290511ae209ac09eac40e
From https://github.com/avidullu/badminton-highlight-indexer
 ! e358b28..69fb2f1  master     -> origin/master  (unable to update local ref)
---BRANCH---
docs/court-floor-rally-end
```

4. user (2026-06-19T16:29:49.543Z)

```
output/GX010128_golden_features.json
output/human_lovo_manifest.json
```

5. assistant (2026-06-19T16:29:52.638Z)

```
The branch differs from what's reported (I'm on `docs/court-floor-rally-end`), and there's a
ref-lock issue but the data files exist. Let me proceed read-only with the local checkout. Let
me examine the key code and data.
```

6. user (2026-06-19T16:29:55.370Z)

```
keys: ['name', 'fps', 'frame_width', 'candidates', 'gts']
n_gt: 52
first 3 gts: [[9.661, 32.173], [40.419, 47.165], [53.165, 57.672]]
last gt: [736.003, 749.009]
```

7. user (2026-06-19T16:29:58.678Z)

```
FOUND entry keys: ['name', 'fps', 'frame_width', 'labels', 'trajectory']
name: GX010128
trajectory: C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/GX010128_june12_traj.csv
fps: 59.94
frame_width: 1920.0
```

8. assistant (2026-06-19T16:30:01.366Z)

```
n_gt=52 confirmed. Let me check the trajectory file exists and read the key code modules.
```

9. user (2026-06-19T16:30:03.437Z)

```
-rw-r--r-- 1 avidu 197609 1543847 Jun 16 19:04 C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/GX010128_june12_traj.csv
---HEAD---
Frame,Visibility,X,Y
0,1,881.1199951171875,585.0825691223145
1,1,877.6984405517578,595.3759574890137
```

10. user (2026-06-19T16:30:03.592Z)

```
1       """Prominent-court shuttle gate ? multicourt G2 (see
``docs/PROMINENT_COURT_DETECTION.md``).
2
3       In multicourt footage the WASB shuttle tracker follows ANY shuttle in frame, so a rally
on a
4       background / adjacent court becomes a false rally on the filmed match. This gate re-
defines the
5       shuttle signal ``(x, y, visible)`` ? ``(x, y, visible AND inside-the-prominent-court)``:
a visible
6       shuttle point whose normalised position is outside the prominent-court polygon is marked
NOT
7       visible, so the velocity windower never opens a rally there.
8
9       Pure (no GPU / cv2): point-in-polygon over the court polygon (reuses
10      ``fusion_features.point_in_polygon``). **Subtractive only** ? it can only CLEAR
visibility, never
11      set it ? so on a correct polygon it suppresses out-of-court rallies without
manufacturing any.
12      ``polygon=None`` (or < 3 vertices, or non-positive frame dims) ? an EXACT no-op (single-
court /
13      unconfigured footage), matching ``rally_gate``'s do-no-harm discipline.
14
15      The polygon is the court FLOOR in normalised 0-1 image coords (same convention as
16      ``FusionConfig.court_polygon``). ``height_pad`` is the MVP pseudo-3D play-volume (§4.3):
```

```
a point up
17      to ``height_pad`` (normalised) ABOVE the floor ? i.e. higher in frame, smaller y ? is
still
18      admitted, so a high clear over the prominent court is not clipped.
19      """
20      from __future__ import annotations
21
22      from dataclasses import replace
23      from typing import Any, List, Optional, Sequence, Tuple
24
25      from backend.pipeline.detectors.fusion_features import point_in_polygon
26
27      Polygon = Sequence[Tuple[float, float]]
28
29
30      def admitted(nx: float, ny: float, polygon: Polygon, height_pad: float = 0.0) -> bool:
31          """Is the normalised point inside the court floor polygon, OR within ``height_pad``
above it?
32
33          The pseudo-3D air column: shifting the point DOWN by ``height_pad`` (toward the
floor) and
34          finding it inside means the original sits within ``height_pad`` above the floor ? a
high shuttle
35          over the prominent court, admit it."""
36          if point_in_polygon((nx, ny), polygon):
37              return True
38          return height_pad > 0.0 and point_in_polygon((nx, ny + height_pad), polygon)
39
40
41      def in_court_fraction(interval: Tuple[float, float], points: List[Any], polygon:
Polygon,
42                            fps: float, frame_width: float, frame_height: float,
43                            height_pad: float = 0.0) -> float:
44          """Fraction of a candidate rally's VISIBLE shuttle points that lie inside the
prominent court.
45
46          ``interval`` is ``(start_s, end_s)``; points carry ``.frame``. Returns 1.0 when
there are no
47          visible points in the interval (a window with no shuttle is not an out-of-court
false positive ?
48          leave that judgement to the windower)."""
49          if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
50              return 1.0
51          fs, fe = interval[0] * fps, interval[1] * fps
52          n = inside = 0
53          for p in points:
54              if getattr(p, "visible", False) and fs <= p.frame <= fe:
55                  n += 1
56                  if admitted(p.x / frame_width, p.y / frame_height, polygon, height_pad):
57                      inside += 1
58          return 1.0 if n == 0 else inside / n
59
60
61      def gate_intervals_to_court(intervals: List[Tuple[float, float]], points: List[Any],
62                                  polygon: Optional[Polygon], fps: float,
63                                  frame_width: float, frame_height: float,
64                                  min_in_court_frac: float = 0.5, height_pad: float = 0.0
65                                  ) -> List[Tuple[float, float]]:
66          """Drop candidate rallies whose shuttle is MAJORITY out-of-court ? the window-level
gate.
67
68          Point-level gating (:func:`gate_points_to_court`) is leaky: a background-court rally
whose shuttle
69          occasionally drifts into the prominent court keeps a few in-court points, enough to
```

```
hold a (sparse)
70          window (measured: GX010142 rally #1 is 90% adjacent-court yet a 10% central tail
survives a hard
71          point gate). This drops an interval whose in-court point fraction is below
``min_in_court_frac``,
72          so a rally that is *mostly* in the adjacent court is removed wholesale. No-op
(returns all
73          intervals) when ``polygon`` is None/degenerate. Subtractive-only."""
74          if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
75              return intervals
76          return [iv for iv in intervals
77                  if in_court_fraction(iv, points, polygon, fps, frame_width, frame_height,
height_pad)
78                  >= min_in_court_frac]
79
80
81      def gate_points_to_court(points: List[Any], polygon: Optional[Polygon],
82                               frame_width: float, frame_height: float,
83                               height_pad: float = 0.0) -> List[Any]:
84          """Return ``points`` with visibility CLEARED for any visible shuttle outside the
prominent-court
85          polygon (expanded upward by ``height_pad``).
86
87          No-op (returns the input list unchanged) when ``polygon`` is None/degenerate or the
frame dims
88          are non-positive. Otherwise each gated point is a NEW ``TrajectoryPoint`` (the input
list and
89          its points are never mutated)."""
90          if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0:
91              return points
92          out: List[Any] = []
93          for p in points:
94              if getattr(p, "visible", False):
95                  nx, ny = p.x / frame_width, p.y / frame_height
96                  if not admitted(nx, ny, polygon, height_pad):
97                      p = replace(p, visible=False)
98              out.append(p)
99          return out
100
```

11. assistant (2026-06-19T16:30:05.918Z)

Now let me read the supporting modules to understand the exact APIs.

12. user (2026-06-19T16:30:09.729Z)

```
parse_trajectory_csv sig: (csv_win_path: str) ->
List[backend.pipeline.detectors.tracknet_runner.TrajectoryPoint]
n points: 45298
point attrs: ['frame', 'visible', 'x', 'y']
sample: 0 True 881.1199951171875 585.0825691223145
n visible: 27133
```

13. assistant (2026-06-19T16:30:12.543Z)

27133 visible points confirmed. Now let me read the eval modules and verify the visible-point count matches the reported polygon_summary (27133 visible).

14. user (2026-06-19T16:30:14.523Z)

```
windows_for sig: (video: 'Dict[str, Any]', preset: 'WindowingPreset', min_crossings: 'int' = 0)
-> 'List[Interval]'
SERVED: WindowingPreset(name='served', velocity_thresh=0.02, merge_gap=2.0,
```

```
min_window_duration=2.0, max_window_duration=6.0, hard_cap=False, inactivity_end=1.5,
inactivity_rally_thresh=0.2, inactivity_min_density=0.3, blob_fallback=False, blob_factor=4.0)
<class 'backend.eval.windowing.WindowingPreset'>
```

15. user (2026-06-19T16:30:16.017Z)

```
score sig: (preds: List[Tuple[float, float]], gts: List[Tuple[float, float]], iou_threshold:
float = 0.5, match_result: Optional[backend.eval.metrics.MatchResult] = None) ->
backend.eval.metrics.Score
match_intervals sig: (preds: List[Tuple[float, float]], gts: List[Tuple[float, float]],
iou_threshold: float = 0.5) -> backend.eval.metrics.MatchResult
match_1to1 sig: (preds: 'List[Interval]', gts: 'List[Interval]', tau_start: 'float' = 2.0,
tau_end: 'float' = 1.5) -> 'Tuple[List[Match], float, float, float]'
point_in_polygon sig: (point: 'Tuple[float, float]', polygon: 'Sequence[Tuple[float, float]]')
-> 'bool'
```

16. assistant (2026-06-19T16:30:18.923Z)

Now let me check the structure of `match_intervals` MatchResult to correctly count FPs among dropped rallies.

17. user (2026-06-19T16:30:20.313Z)

```
MatchResult fields: []
@dataclass
class MatchResult:
    """Outcome of matching predictions to ground truth at a given IoU threshold."""
    matches: List[Match] = field(default_factory=list)
    unmatched_pred_indices: List[int] = field(default_factory=list)  # false positives
    unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives (misses)
```

18. assistant (2026-06-19T16:30:22.984Z)

`unmatched_pred_indices` directly gives FP predictions. Now I have everything. Let me run the full baseline computation first.

19. user (2026-06-19T16:30:28.863Z)

```
baseline n_pred: 81
n_gt: 52
base precision: 0.493827
base recall: 0.769231
base f1: 0.601504
n matches: 40
n unmatched_pred (FP): 41
n unmatched_gt (miss): 12
```

20. assistant (2026-06-19T16:30:32.118Z)

Baseline reproduces exactly: n_pred=81, precision=0.493827, recall=0.769231, f1=0.601504, 41 FPs. Now let me independently analyze the shuttle point distribution to derive my own polygon, rather than trusting the reported one.

21. user (2026-06-19T16:30:36.530Z)

```
n visible: 27133
   x: 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9
y=0.0     0     0     9    63    88   306    91     0     1     0
y=0.1     0     0    67   179   234   425   176    41     1     0
y=0.2     0     2    62   151   585   850   869    92     0     0
y=0.3     2    11   150   304  1309  3231  1959   188  1688     0
y=0.4     0    21    54   251  7663  1458  1507   225     7     0
y=0.5    61   148   182   183   218   698  1058     8     0     0
y=0.6    86    83    28    32     2    24     0     0     0     0
```

```
y=0.7     0     0     0     0     0     1     1     0     0     0
y=0.8     0     0     0     0     0     0     0     0     0     0
y=0.9     0     0     0     0     0     0     0     0     0     0
```

22. assistant (2026-06-19T16:30:42.858Z)

This is the critical evidence. Let me analyze it carefully:

- **Dominant mass**: x in [0.4, 0.7], y in [0.2, 0.6] ? the prominent court.
- **Secondary cluster**: there's a tight blob at **x=0.8, y=0.3 ? 1688 points** (the cell `[0.8-0.9), [0.3-0.4)`). This is the reported "adjacent court / stuck tracker" cluster.

Now the key adversarial question: the reported polygon caps x ? 0.78, which excludes that x~0.85 cluster. But the reported result says **0 dropped** ? so excluding it had no effect. Let me verify the polygon's recall-safety AND independently check whether that secondary cluster is actually a separate court vs. just a stuck tracker, AND re-derive the polygon. Let me first validate the reported polygon exactly.

23. user (2026-06-19T16:30:53.511Z)

```
golden rallies with no visible pts: 0
golden medians evaluated: 52
inside: 52 outside: 0
pct_golden_in_polygon (of evaluated): 1.0
pct over ALL 52: 1.0
outside medians: []
```

24. assistant (2026-06-19T16:30:56.872Z)

Reported polygon is recall-safe: all 52 golden medians inside, pct=1.0. Now the critical adversarial check ? run the actual gate and verify the "0 dropped" claim and the min in-court fraction claim of 0.62.

25. user (2026-06-19T16:31:09.558Z)

```
min in-court frac: 0.6231
max in-court frac: 1.0
n windows with frac < 0.5: 0
lowest 10 fracs: [0.623, 0.658, 0.727, 0.769, 0.77, 0.771, 0.81, 0.812, 0.813, 0.82]
gated n_pred: 81
n_dropped: 0
base P/R/F1: 0.493827 0.769231 0.601504
gated P/R/F1: 0.493827 0.769231 0.601504
base_tolf1: 0.466165 gated_tolf1: 0.466165
```

26. assistant (2026-06-19T16:31:15.758Z)

The core claim is confirmed exactly:
- min in-court fraction = **0.6231** (reported "0.62" ?)
- 0 windows below 0.5 threshold ? **0 dropped** ?
- gated == baseline exactly (P/R/F1 unchanged, tolf1 unchanged)
- base_tolf1 = 0.466165 ?, gated_tolf1 = 0.466165 ?

Now the deeper adversarial questions remain. The reported result is a NULL, which is the honest direction. But I need to verify it's not a *masked* null ? i.e., could a recall-safe polygon that more aggressively excludes the secondary cluster actually drop real FPs? The reported polygon caps x?0.78. Let me check: is the x~0.85 secondary cluster actually a separate court, and does any baseline window's shuttle live mostly there? Let me investigate the 1688-point cluster temporally.

27. user (2026-06-19T16:31:22.100Z)

```
secondary cluster pts: 1762
time range (s): 1.0 - 754.3
```

```
sec x: min/median/max: 0.78 0.872 0.875
top 5 coord clusters (rounded to 0.1px): [((1680.0, 354.4), 284), ((1680.0, 356.2), 85),
((1680.0, 348.8), 70), ((1680.0, 352.5), 70), ((1680.0, 354.5), 53)]
distinct seconds spanned by secondary cluster: 291
```

28. assistant (2026-06-19T16:31:29.074Z)

Critical finding: the secondary cluster is a **STUCK TRACKER**, not an adjacent court. Evidence: 284 points all at exactly x=1680.0, y=354.4 (and the top clusters all share x=1680.0). A real adjacent-court rally would have the shuttle moving around; this is the tracker locking onto a fixed pixel (1680, 354). It spans 291 distinct seconds across the whole 1-754s video ? consistent with intermittent false-locks, not a real game on another court.

This actually *strengthens* the null read but slightly changes the characterization: the reported summary calls it "adjacent court / stuck tracker," which hedges correctly. Since it's a stuck point (not a moving rally), it never accumulates a velocity-based window ? which is exactly why 0 are dropped. Let me confirm: do any baseline windows overlap this stuck-point region in time, and what is their in-court frac?

29. user (2026-06-19T16:31:40.517Z)

```
n FP windows: 41
FP window in-court fracs (lowest 10):
  frac 0.623 iv 271.2 - 276.9
  frac 0.658 iv 627.4 - 633.2
  frac 0.769 iv 411.6 - 421.8
  frac 0.869 iv 0.0 - 4.2
  frac 0.87 iv 374.0 - 376.6
  frac 0.884 iv 392.8 - 395.7
  frac 0.918 iv 121.7 - 124.5
  frac 0.92 iv 277.5 - 280.0
  frac 0.921 iv 299.7 - 304.1
  frac 0.933 iv 343.8 - 346.6
min FP frac: 0.623
thr=0.5: FP dropped=0, TP dropped=0
thr=0.6: FP dropped=0, TP dropped=0
thr=0.65: FP dropped=1, TP dropped=0
thr=0.7: FP dropped=2, TP dropped=0
thr=0.75: FP dropped=2, TP dropped=1
thr=0.8: FP dropped=3, TP dropped=3
```

30. assistant (2026-06-19T16:31:47.253Z)

This fully corroborates the null at the prescribed threshold (0.5) and adds important nuance:

- At the **prescribed min_in_court_frac=0.5**: 0 FPs dropped, 0 TPs dropped ? exact no-op. **The reported result is correct.**
- Even at threshold 0.6: still 0 dropped. The minimum FP in-court fraction is 0.623 ? every single one of the 81 windows is majority-in-court, so the gate at 0.5 cannot fire on any window. This is the honest reason for the null.
- (Sidebar: pushing the threshold well above the prescribed 0.5 to 0.7 would catch 2 FPs cleanly, but at 0.75+ it starts taking real TPs ? confirming the gate at the specified 0.5 setting is genuinely a no-op here, not a tunable hidden win.)

Now let me verify the remaining reported numbers I haven't independently confirmed: the visible-point counts in the polygon_summary (27133 total, 2070 excluded, secondary ~1685/2070).

31. user (2026-06-19T16:31:53.500Z)

```
n visible: 27133
visible excluded by polygon: 2070
visible retained: 25063
secondary cluster (x>0.78, y in 0.30-0.40): 1712
```

32. assistant (2026-06-19T16:31:59.426Z)

All counts confirmed:
- 27133 visible points ? (reported)
- 2070 visible points excluded by the polygon ? (reported "Excludes 2070 visible pts")
- Secondary cluster ~1712 (reported ~1685, close ? the small diff is just the exact y-band cutoff) ?

Every number in the reported row checks out exactly. Let me do a final consolidated re-run to lock all output fields together with no rounding drift.

33. user (2026-06-19T16:32:14.208Z)


```
{
 "name": "GX010128",
 "n_gt": 52,
 "pct_golden_in_polygon": 1.0,
 "base_n_pred": 81,
 "base_precision": 0.493827,
 "base_recall": 0.769231,
 "base_f1": 0.601504,
 "base_tolf1": 0.466165,
 "gated_n_pred": 81,
 "gated_precision": 0.493827,
 "gated_recall": 0.769231,
 "gated_f1": 0.601504,
 "gated_tolf1": 0.466165,
 "n_dropped": 0,
 "delta_precision": 0.0,
 "delta_recall": 0.0,
 "delta_f1": 0.0
}
```


34. assistant (2026-06-19T16:32:27.226Z)

Every field is independently reproduced exactly. Let me confirm the `n_dropped_were_fp` field. Since 0 rallies were dropped, by definition 0 dropped were FPs.

The verification is complete. Summary of my adversarial findings:

All reported numbers reproduce exactly (no inflation):
- n_gt=52, base/gated n_pred=81, P=0.493827, R=0.769231, F1=0.601504, tolF1=0.466165 ? all confirmed.
- delta_precision/recall/f1 = 0, n_dropped=0, n_dropped_were_fp=0 ? confirmed.

Adversarial checks passed:
1. **Polygon recall-safe**: all 52/52 golden-rally medians fall inside the polygon ? pct=1.0. Not gamed ? even the loosest reading gives 100%.
2. **Secondary cluster genuinely excluded, but it's a STUCK TRACKER, not an adjacent court**: the ~1762 excluded points cluster at exactly x=1680px (284 pts at the identical pixel), spanning 291 distinct seconds across the whole 754s clip ? a fixed-pixel false-lock, not a moving rally. The reported "adjacent court / stuck tracker" hedge is accurate; "stuck tracker" is the more precise characterization.
3. **The null is real, not masked**: the minimum in-court fraction across all 81 baseline windows is 0.6231 ? every window is majority-in-court, so the 0.5 gate cannot fire on any. The 41 FPs are real over-fire (seg_ratio 81/52=1.56) but none are out-of-court-majority. The stuck-pixel points never accumulate into a velocity window, so they never form a droppable rally.
4. **Recall not silently lost**: gated == baseline interval-for-interval; recall delta is a true 0, not a hidden drop.
5. Polygon-exclusion counts confirmed: 27133 visible, 2070 excluded (reported 2070), secondary band ~1712 (reported ~1685, matches within the y-band cutoff).

Verdict: the reported row is CORRECT and not inflated. It is an honest null. The one refinement I'd make to the `read` is to characterize the secondary cluster as a stuck-tracker pixel-lock (the dead-giveaway being 284 points at exactly x=1680px) rather than leaving

"adjacent court" as the leading interpretation ? but this does not change any metric.

35. user (2026-06-19T16:32:39.463Z)

Structured output provided successfully

36. assistant (2026-06-19T16:32:44.312Z)

Verified. The reported row is correct and not inflated ? an honest null. Every metric reproduced exactly against the local checkout, the polygon is recall-safe (52/52 golden medians inside), recall was not silently lost, and the null is genuine (every baseline window is ?0.62 in-court so the 0.5 gate cannot fire). The one substantive refinement: the excluded secondary cluster is a stuck-tracker pixel-lock (284 points at exactly x=1680px) rather than a live adjacent court ? which is the real reason there's no out-of-court rally to drop.